AI-First Development

Open-Ended Workshop

Choose Your Project • Build with AI • 3 Hours

Workshop Agenda (3 Hours)

- Part 1: Foundation & Project Selection (45 min)

  - AI-First philosophy quick recap
  - Review sample projects from unveiling-claude
  - Choose your project (or bring your own idea)
  - Initial setup

- Part 2: Independent Build (90 min)

  - Planning phase with AI
  - Implementation sprints (with break)
  - Work at your own pace

- Part 3: Review & Share (45 min)

- Git workflow - commit and PR
- Show & Tell - demo your progress
- Wrap-up and next steps

Part 1: Foundation & Project Selection

AI-First Philosophy (Quick Recap)

- AI executes 100% → Human validates 100%

Core principle:

- AI handles coding, testing, documentation
- Human handles validation, review, decisions

- You are the architect, AI is the builder
- Today: You choose what to build!

The 10-Step Workflow (Reference)

- 1. Specification (Human writes)
- 2. Database Schema (AI → Human reviews)
- 3. Repository Layer (AI → Human reviews)
- 4. API Endpoints (AI → Human reviews)
- 5. API Tests (AI → Human verifies)
- 6. Frontend (AI → Human reviews)
- 7. UI Tests (AI → Human verifies)
- 8. Documentation (AI → Human reviews)
- 9. Code Review (Human)
- 10. Deployment (AI → Human verifies)

- Use as much or as little as fits your project

Today's Format

Different from guided workshops:

- YOU choose the project
- YOU decide what to build
- YOU work at your own pace

Instructor is here to:

- Help when you're stuck
- Answer questions
- Keep time

• Goal: Experience real AI-first development

Sample Projects Overview

Sample Specs: github.com/emmanuelandre/unveiling-claude

- Repository contains several project specifications

Each project has:

- Detailed specifications
- Architecture documents
- Implementation guidance

- Review the specs before choosing
- Or: Bring your own project idea!

Option 1: manu-code (High Complexity)

• AI-powered CLI code generation assistant

Features:

■ Natural language to code

■ Multi-provider support (Anthropic, OpenAI, Gemini)

■ File operations, shell execution, git integration

Tech Stack:

■ TypeScript / Node.js

■ Commander, Inquirer CLI libraries

• Good for: Experienced developers wanting a challenge

• Spec: manu-code/manu-code-specs.md

Option 2: task-manager (Medium Complexity)

• Task management system with API

Features:

■ CRUD operations for tasks

■ User authentication

■ Task assignment and status tracking

Tech Stack:

■ Go or Node.js backend

■ PostgreSQL database

• Good for: Learning full-stack API development

• Spec: task-manager/ directory

Option 3: my-api-project (Low Complexity)

• Simple REST API starter project

Features:

■ Basic CRUD endpoints

■ Database migrations

■ Simple authentication

Tech Stack:

■ Go backend

■ PostgreSQL or SQLite

• Good for: Beginners or quick projects

• Spec: my-api-project/ directory

Option 4: Other Projects

prompt-ops:

- Prompt operations tooling
- Medium complexity

ui-to-test:

- UI testing project
- Focus on frontend testing

Bring Your Own Idea:

- Have a project in mind? Build it!
- Personal tool, side project, experiment
- Tell your AI assistant what you want to build

How to Review a Spec

When looking at a spec, check:

- What problem does it solve?
- What are the core features?
- What tech stack is suggested?
- What can I realistically build in 90 minutes?

Scoping for today:

- Pick ONE feature to implement
- Focus on core functionality
- Tests are important but scope them too

■ Hands-On: Exercise 1: Browse & Choose Your Project

YOUR PROMPT:

```
Go to: github.com/emmanuelandre/unveiling-claude

Browse the available projects:
- manu-code/ - AI CLI assistant (High complexity)
- task-manager/ - Task management API (Medium)
- my-api-project/ - Simple API starter (Low)
- prompt-ops/ - Prompt operations (Medium)
- ui-to-test/ - UI testing (Medium)

Read the specs and choose ONE project.

OR: Come up with your own idea!

When ready, raise your hand to share your choice.
```

What You Should See:

- You've chosen a project and have a rough idea of what to build

Decision Checklist

Before moving on, confirm:

- ■ ■ I've chosen a project
- ■ ■ I know which feature(s) to focus on
- ■ ■ I have a rough idea of the tech stack
- ■ ■ I'm ready to start!

If stuck:

- ■ Start with my-api-project (simplest)
- ■ Ask instructor for guidance
- ■ Pair with someone on the same project

Hands-On: Exercise 2: Initial Setup

YOUR PROMPT:

```
Choose ONE option:

OPTION A: Fork unveiling-claude (if using sample project)
1. Go to github.com/emmanuelandre/unveiling-claude
2. Click "Fork" to create your copy
3. Clone your fork:
git clone https://github.com/[YOUR-USERNAME]/unveiling-claude.git
cd unveiling-claude/[your-project]

OPTION B: Create new repo (if using own idea)
1. Create new repo on GitHub
2. Clone it locally:
git clone https://github.com/[YOUR-USERNAME]/[your-repo].git
cd [your-repo]
git checkout -b feature/initial-setup

Open your AI assistant and get ready!
```

What You Should See:

- Repository set up and ready to work

Part 2: Independent Build (90 min)

How This Part Works

You will work independently:

- Use your AI assistant as your coding partner
- Follow the 10-step workflow (as much as applies)
- Ask instructor if completely stuck

Timeline:

- Planning Phase: 20 min
- Implementation Sprint 1: 35 min
- Break: 10 min
- Implementation Sprint 2: 25 min

• Checkpoints will be announced

Planning Phase (20 min)

■ Hands-On: Exercise 3: Create Your Plan with AI

YOUR PROMPT:

```
Tell your AI assistant:

I want to build [PROJECT NAME].

Project context: [Brief description or link to spec]

Help me create:
1. A CLAUDE.md (or project rules file) with:
- Project overview
- Tech stack
- Project structure
- Commands (build, test, run)
- Testing requirements

2. A specification for the FIRST feature I should build
- Keep it scoped for ~60 min implementation
- Include success criteria
```

```
3. A rough implementation plan

My tech stack preference: [Go/Node/Python/TypeScript]
```

What You Should See:

• CLAUDE.md created and first feature specified

Planning Tips

Keep scope realistic:

- One feature, working end-to-end
- Better to finish small than abandon big

Be specific with AI:

- Describe what you want clearly
- Include constraints and preferences
- Ask for clarification if needed

Example good scope:

- User registration endpoint with validation
- Single CRUD resource with tests
- CLI command that does one thing well

Checkpoint: Share Your Plan

In 2 minutes, be ready to share:

- ■ What project did you choose?
- ■ What feature are you building?
- ■ What's your first step?

- Quick round-robin (30 seconds each)
- This helps everyone stay on track

Implementation Guidelines

Follow the workflow:

- Database/data layer first (if needed)
- Core logic/API second
- Tests third
- Don't skip tests!

Working with AI:

- Review what AI generates
- Ask for explanations if unclear
- Request changes if needed

When stuck:

- Ask AI to explain the error

- Try a different approach
- Ask instructor for help

Quality Checkpoints

After each component, verify:

- ■ ■ Does it compile/run?
- ■ ■ Does it do what the spec says?
- ■ ■ Are there obvious bugs?
- ■ ■ Is the code readable?

Don't worry about:

- ■ Perfect code on first try
- ■ Complete feature coverage
- ■ Production-ready polish

• Focus on: Working software you understand

Sprint 1: Build Core Feature (35 min)

■ Hands-On: Exercise 4: Implementation Sprint 1

YOUR PROMPT:

```
Time to build! Follow your plan.

Suggested approach:
1. Start with data layer (database schema, models)
2. Then business logic (repository, handlers)
3. Then tests

Example prompt to start:
"Based on our spec, let's start implementing.
First, create the database schema for [your feature].
Include proper types, indexes, and constraints."

Continue from there. Work at your own pace.

Checkpoint in 35 minutes!
```

What You Should See:

- Core feature partially or fully implemented

Sprint 1 Checkpoint Questions

Where are you?

- ■ Database/models done?
- ■ Core logic started?
- ■ Any blockers?

Quick assessment:

- ■ On track → Keep going
- ■ Behind → Scope down
- ■ Stuck → Ask for help

• No judgment - everyone works at different speeds

Break (10 min)

Sprint 2: Continue Building (25 min)

Hands-On: Exercise 5: Implementation Sprint 2

YOUR PROMPT:

```
Continue building your feature.

Focus on:
- Completing what you started
- Adding tests for what works
- Making it demo-able

If you finished early:
- Add another small feature
- Improve tests
- Clean up code
- Help a neighbor

Remember: Something working > Something perfect

Checkpoint in 25 minutes - prepare for demo!
```

What You Should See:

- Feature implemented and ready to demo

Prepare for Demo

In your demo, plan to show:

- What you built (quick walkthrough)
- One working feature in action
- What you learned

Keep it short:

- 2-3 minutes max per person
- Focus on the interesting parts
- It's OK if it's not finished!

Part 3: Review & Share

Git Workflow

Before demo, commit your work:

- Stage changes: git add .
- Review: git status
- Commit with conventional format

Commit message format:

- feat(scope): description
- Example: feat(auth): add user registration endpoint

• Push to your fork/repo

Hands-On: Exercise 6: Commit and Push

YOUR PROMPT:

```
Commit and push your work:

1. Review changes:
git status
git diff

2. Stage and commit:
git add .
git commit -m "feat([scope]): [what you built]

- [Key thing 1]
- [Key thing 2]
- [Key thing 3]"

3. Push:
git push origin [your-branch]

4. (Optional) Create PR:
```

```
gh pr create --title "feat: [your feature]" \
--body "## What I Built
[Description]

## What Works
- [Feature 1]
- [Feature 2]"

Or create PR through GitHub web interface.
```

What You Should See:

• Code committed and pushed to GitHub

PR Description Template

Include:

- ## What I Built - brief description
- ## What Works - list working features
- ## What's Next - future improvements
- ## Lessons Learned - optional but valuable

• This documents your workshop experience

• Great reference for continuing at home

Show & Tell (20 min)

Demo Format

Each person: 2-3 minutes

- What project did you choose?
- What did you build?
- Quick demo of one thing working
- What was interesting/challenging?

It's OK to show:

- Partially working features
- Interesting failures
- What you learned from errors

• Celebrate progress, not perfection!

While Others Demo

Listen for:

- Interesting approaches
- Useful AI prompts
- Problems you also faced

- Save questions for after all demos
- Note ideas for your own projects

Wrap-Up

Key Learnings

Today you experienced:

- Choosing and scoping a project
- Planning with AI assistance
- Independent implementation
- Real development workflow

The AI-First approach:

- AI helps you move faster
- You make the decisions
- Review everything
- Tests matter

Continue at Home

Your project isn't finished - keep going!

- Complete the feature you started
- Add more tests
- Try another feature from the spec

Tips for solo work:

- Commit frequently
- Take breaks
- Review AI output carefully
- Document what works

Resources

Sample Specs:

- github.com/emmanuelandre/unveiling-claude

AI Tools:

- Claude Code: claude.ai/code
- Windsurf: codeium.com/windsurf
- Cursor: cursor.sh

Documentation:

- See /docs folder for detailed guides
- CLAUDE.md templates in /examples

Thank You!

Keep Building!